Build Brief: Multidimensional Wheel of Life Assessment

Act as a senior product designer and full-stack engineer. Build a clean, minimal, responsive web application for the Multidimensional Wheel of Life Assessment.

Before writing code, review the project documents listed below, understand how they relate to one another, and propose a short implementation plan. Do not invent missing assessment content or change approved scoring rules without my approval.

1. Documents to reference

Document 1: Product Requirements Document (PRD)

Purpose: Defines what the product must do.

Use the PRD as the primary reference for:

- Product vision and objectives.
- Target users and their needs.
- User journey from welcome page to final results.
- Functional requirements and acceptance criteria.
- Assessment completion rules.
- Results dashboard and radar chart requirements.
- Privacy, security, accessibility, and testing requirements.
- Delivery phases and definition of done.

Treat the PRD as the source of truth for product behaviour. Where a decision is marked unresolved, identify it before implementation rather than making assumptions.

Document 2: Assessment Question Bank

Purpose: Defines what the assessment measures.

Use the Assessment Question Bank as the source of truth for:

- The 12 life paths and their order.
- The dimensions measured within each path.
- The 10 assessment questions under each path.
- The intended relationship between questions and dimensions.
- The proposed response scale and scoring notes.
- Assessment methodology and review considerations.

Preserve the approved question wording. Do not paraphrase, replace, omit, or invent questions without permission.

The question bank is a working draft. Flag any unresolved methodological issues, including the response scale, questions that may not apply to every participant, and provisional interpretation bands.

How the documents work together

The PRD governs product behaviour and technical requirements. The Assessment Question Bank governs assessment content.

Neither document should silently override the other. If you identify a conflict, explain it and recommend a resolution before implementing the affected feature.

2. Product experience

Build the application around this flow:

1. Welcome page introducing the assessment.
2. Brief instructions explaining how to answer.
3. Sequential assessment through the 12 life paths.
4. Automatic scoring in the background.
5. Completion screen and final results reveal.
6. Results dashboard with a 12-axis radar chart, domain scores, interpretation bands, and development insights.

Each path contains 10 questions according to the approved question bank.

Users must not see their calculated domain scores while completing the assessment. Show only their progress through the questions and paths.

The final results must become available only when all required questions have been answered.

3. Scoring and results

Implement scoring as a reusable, independently tested function.

If the approved scoring model is a 1–10 scale, calculate each path's score as the arithmetic mean of its 10 valid responses.

Display scores to one decimal place. Use unrounded scores to determine interpretation bands.

The provisional bands are:

- Needs Attention: 1.0–3.9
- Developing: 4.0–6.9
- Strong Foundation: 7.0–10.0

These bands must remain configurable and must not be treated as validated thresholds.

Do not produce an overall life score by combining all 12 paths.

The radar chart must accurately represent the calculated score for each path. Provide an accessible text-based alternative to the chart.

If the response scale or scoring model has not yet been approved, identify this as a blocker and do not hard-code it as a final decision.

4. Visual design direction

The design must be simple, minimal, calm, and purposeful.

Visual principles

- Clean layouts with generous whitespace.
- Clear typography and readable text.
- Restrained use of colour.
- Neutral backgrounds with one subtle accent colour.
- Thin borders and understated dividers.
- Simple, consistent buttons and form controls.
- Minimal decorative elements.
- Clear visual hierarchy.
- Responsive layouts for mobile, tablet, and desktop.

Avoid:

- Cluttered dashboards.
- Excessive cards, gradients, shadows, or animations.
- Bright, distracting colours.
- Unnecessary illustrations and decorative icons.
- Gamification, badges, streaks, or competitive rankings.
- Dense question screens.
- Generic, overly elaborate AI-generated interface styling.

The assessment should feel reflective, professional, trustworthy, and easy to use.

Question interface

Display one question at a time, with the path name, a concise progress indicator, the question text, and the response control.

Keep the response scale easy to understand and operate. Make selected answers visually clear.

Provide simple Previous and Next controls. Preserve responses when users navigate backwards.

Do not display scores, performance indicators, or feedback that could influence answers to subsequent questions.

Results interface

Use a restrained, readable results layout.

Place the radar chart prominently, followed by a clear breakdown of all 12 paths.

Make lower-scoring areas and stronger areas distinguishable without using alarming colours or judgemental language.

Include concise explanations and reflection prompts where supported by the approved assessment content. Do not generate unsupported personal conclusions.

5. Technical implementation

Before choosing the stack, inspect the existing project. If a codebase already exists, preserve its working structure unless there is a clear reason to change it.

Separate the application into maintainable modules:

- Domain and question configuration.
- Response capture and validation.
- Scoring and interpretation.
- Assessment navigation and progress.
- Results generation and radar chart.
- Data persistence and access control, where required.

Avoid duplicating question content across components.

The question bank should be represented in a structured format so that questions can be maintained independently of the UI.

Implement and test the scoring logic before building the complete assessment interface.

Do not rely on CSS or client-side hiding to protect scores. If score confidentiality is required before completion, enforce result access through an appropriate trusted backend.

Do not introduce accounts, payments, analytics, AI-generated feedback, or other features that are not required by the approved scope.

6. Build and verification process

Work in the following order:

Phase 1: Review

- Read both documents completely.
- Confirm the 12 paths and question counts.
- Identify conflicting or unresolved requirements.
- Propose the technical architecture and minimal design direction.

Phase 2: Foundation

- Establish the project structure and visual styles.
- Implement the question and domain data model.
- Implement scoring and automated tests.

Phase 3: Assessment

- Build the welcome and instructions pages.
- Implement question navigation and progress tracking.
- Add all approved assessment questions.
- Enforce completion rules and hidden interim scores.

Phase 4: Results

- Build the radar chart.

- Display all 12 calculated scores and interpretation bands.
- Add evidence-based summaries and reflection prompts.

Phase 5: Quality assurance

- Test calculations and score boundaries.
- Test incomplete submissions and navigation.
- Verify that results cannot be accessed prematurely.
- Test mobile and desktop layouts.
- Check accessibility and privacy controls.

After each phase, summarise what was implemented, what was tested, and any outstanding decisions.

7. Definition of success

The application is successful when users can complete the assessment through a straightforward interface, their scores are calculated correctly, no interim scores are revealed, and the final results provide a clear and accurate visual summary of all 12 life paths.

The design must remain minimal throughout. Prioritise clarity, assessment integrity, and ease of use over visual complexity.

Begin by reviewing the two documents and presenting your implementation plan. Do not begin the full build until I approve the plan.